Cache replacement policies

Sparsh Mittal

Cache hierarchy in commercial processors

	Intel Nehalem	AMD Opteron X4
L1 caches (per core)	L1 I-cache: 32KB, 64-byte blocks, 4-way, approx LRU replacement, hit time n/a L1 D-cache: 32KB, 64-byte blocks, 8-way, approx LRU replacement, write-back, hit time n/a	L1 I-cache: 32KB, 64-byte blocks, 2-way, LRU replacement, hit time 3 cycles L1 D-cache: 32KB, 64-byte blocks, 2-way, LRU replacement, write-back, hit time 9 cycles
L2 unified cache (per core)	256KB, 64-byte blocks, 8-way, approx LRU replacement, write-back, hit time n/a	512KB, 64-byte blocks, 16-way, approx LRU replacement, write-back, hit time n/a
L3 unified cache (shared)	8MB, 64-byte blocks, 16-way, replacement n/a, write-back, hit time n/a	2MB, 64-byte blocks, 32-way, replace block shared by fewest cores, write-back, hit time 32 cycles

n/a: data not available

What is a cache replacement operation

Assume that an L1 cache set has N ways. All of them contain valid data blocks

The cache receives a request for a data block which is not present (i.e., miss).

This data will be searched in lower-cache (L2). Assume it hits in L2 cache.

The data will be copied to L1 cache (and also provided to the core).

But all the ways of that set already have valid blocks. So, one of them needs to be replaced.

Here is where a replacement operation is required.

Which cache requires a replacement policy

Direct mapped cache: no need of replacement policy since there is only one cache way

Set associative (or fully-associative) cache

- Prefer non-valid entry, if there is one
- Otherwise, choose among entries in the set

Widely used replacement policies

Random

Randomly select an existing way and replace that

FIFO

Block which first came in cache should be evicted first

without any regard to how often or how many times they were accessed before.

Least-recently used (LRU): Choose the one unused for the longest time

Simple for 2-way, manageable for 4-way, too hard beyond that

Optimal: replaces the element which will be accessed farthest in future.

MRU = most-recently used

Understanding FIFO and LRU policies

Consider a single set of a 4-way cache, which sees following accesses: P, Q, R, S, P, U.
Show cache contents (and replacements) with both FIFO and LRU policy.

FIFO policy

P	P	0							Miss
Q	P	0	Q	1					Miss
R	P	0	Q	1	R	2			Miss
S	P	0	Q	1	R	2	S	3	Miss
P	P	0	Q	1	R	2	S	3	Hit
U	U	3	Q	0	R	1	S	2	Miss.

Counter: 3=Latest-Inserted, 0=Oldest inserted
Counters are updated only when block is inserted in cache; not on access

Summary: Replace in the order in which blocks came in cache

LRU policy

P	P	0							Miss
Q	P	1	Q	0					Miss
R	P	2	Q	1	R	0			Miss
S	P	3	Q	2	R	1	S	0	Miss
P	P	0	Q	3	R	2	S	1	Hit
U	P	1	U	0	R	3	S	2	Miss.

Counter: 3=LRU block, 0=MRU block
Counters are updated both when block is inserted in cache and when an access happens

Summary: Replace in the order in which blocks were accessed (used).

Understanding OPTIMAL policy

Consider a single set of a 4-way cache, which sees following accesses: P, Q, R, S, U, P, Z, Q, S, R.
Show cache contents (and replacements) with both FIFO and LRU policy.

OPTIMAL policy

P	P				Miss
Q	Q	P			Miss
R	R	Q	P		Miss
S	S	R	Q	P	Miss
U	S	U	Q	P	Miss

Continues like this……..

Access to U misses. Which one to replace?
Out of S, R, Q and P, we see that R comes farthest in future.
Hence, we replace R.

Solved example on optimal policy

OPTIMAL policy can be implemented only if you know the future, i.e., you know whole access sequence.

In programming assignment, we will give you the entire access sequence, so you can implement OPTIMAL policy, but in online execution, this policy cannot be implemented.

Solved example on optimal policy

Consider below access sequence coming to a 3-way fully-associative cache. Show cache state after each access and which access leads to a miss.

1	2	3	2	1	5	2	1	6	2	5	6	3	1	3	6	1	2	4	3

Solved example on optimal policy

1	2	3	2	1	5	2	1	6	2	5	6	3	1	3	6	1	2	4	3

1	1	1	1	1	1	1	1	6	6	6	6	6	6	6	6	6	2	2	2
	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	4	4
		3	3	3	5	5	5	5	5	5	5	3	3	3	3	3	3	3	3
*	*	*			*			*				*	*				*	*	

* = miss